

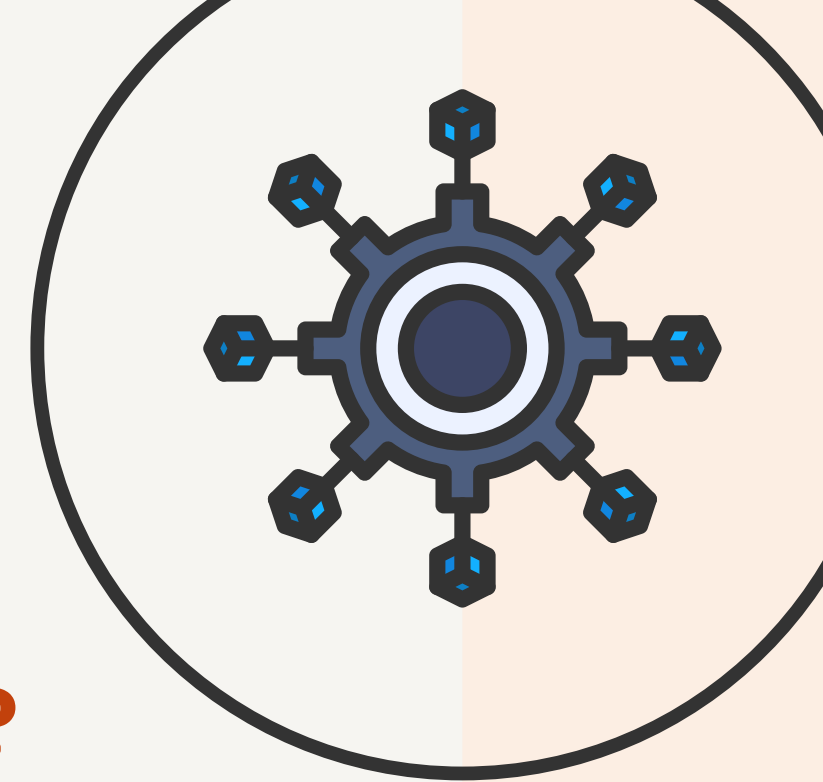
TALLER 01 · ARQUITECTURA DE SOFTWARE

Chat por canales

Microservicios con Flutter, Go, gRPC y Cassandra

Angy Bautista, Miguel Laiton, Sebastian Sánchez

Microservicios



¿Qué es?

- Servicios pequeños y autónomos, uno por capacidad de negocio.
- Cada servicio es dueño de sus datos.
- Se despliegan y escalan por separado.
- Se comunican por mecanismos ligeros (REST, gRPC, eventos).

¿Qué no es?

- No es un monolito distribuido: Servicios pequeños que dependen unos de otros para compilar o desplegarse.
- No es compartir base de datos: Múltiples servicios pequeños que dependen de una sola base de datos centralizada.
- No es una tecnología: No depende de usar Docker, Kubernetes o un lenguaje en específico, sino de cómo se diseña el software.

Clasificación del estilo

← Una sola unidad desplegable

Muchas unidades desplegables →

Monolito en capas

Un despliegue,
partido por
capas técnicas

Monolito modular

Un despliegue,
partido por
dominio

Basado en servicios

Pocos
servicios
grandes, base
compartida

SOA

Servicios de
empresa unidos
por un bus
central (ESB)

Microservicios

Muchos servicios
pequeños, cada
uno con su base

Clasificación: estilo **distribuido, particionado por dominio**, de la familia de los estilos orientados a servicios.

Características principales de los microservicios



01

Servicios como componentes

No son librerías dentro de un monolito: cada uno corre como un proceso propio.



02

Por capacidad de negocio

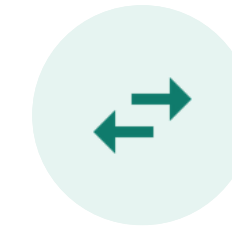
Cada servicio responde a un dominio del negocio, como pagos o pedidos.



03

Despliegue independiente

Se actualiza y escala un servicio sin tocar ni detener al resto.



04

Extremos inteligentes, canales simples

La lógica vive en los servicios; se comunican por REST, gRPC o eventos.



05

Datos descentralizados

Una base de datos por servicio: cada uno es dueño de sus datos.



06

Libertad tecnológica

Cada equipo elige el lenguaje o framework que mejor le sirve.



07

Infraestructura automatizada

CI/CD, contenedores y orquestación hacen viable operar muchos servicios.



08

Diseño para el fallo

Se asume que un servicio puede caer y el sistema debe seguir funcionando.

Idea central: piezas pequeñas, independientes y automatizadas que se combinan para formar un sistema.

Historia Microservicios

1968

Ley de Conway:
el sistema copia
la estructura del
equipo

2000s

SOA y web
services, con
buses centrales
pesados

2008–16

Netflix migra a
cientos de
servicios en la
nube

2014

Lewis y Fowler
definen el
término

2014+

Docker y
Kubernetes
abaratan
desplegar
muchos
servicios

2018–23

Segment, Uber
y Prime Video
consolidan
servicios

Lo que se gana y lo que se pierde

Ventajas

- Despliegue independiente: Actualiza un servicio sin tocar el resto.
- Escalabilidad selectiva: Escala solo los servicios con mayor demanda.
- Libertad tecnológica: Usa el lenguaje o framework ideal para cada servicio.
- Aislamiento de fallos: Si un servicio cae, el resto sigue funcionando.
- Equipos autónomos: Un equipo es dueño de su producto de principio a fin.

Desventajas

- Complejidad distribuida: Mayor reto en redes, latencia y consistencia de datos.
- Alta exigencia operativa: Exige automatización avanzada (CI/CD, Kubernetes, monitoreo).
- Costos elevados: Mantener múltiples servicios y bases de datos es más caro.
- Depuración compleja: Rastrear un error que pasa por varios servicios es difícil.
- Riesgo de caos: La libertad tecnológica mal manejada genera fragmentación.

Cada problema tiene su patrón

Problema común	Patrón de Diseño
El cliente conoce demasiados servicios (las apps o clientes no saben a qué IP o URL conectarse directamente)	API Gateway (un punto de entrada único que enruta las peticiones de forma segura)
Servicios acoplados por tablas compartidas (varios microservicios leen y escriben en la misma base de datos)	Base de datos por servicio (cada microservicio gestiona su propio almacenamiento de forma exclusiva)
Un servicio lento arrastra a los demás (si un servicio falla o se congela, bloquea todo el hilo de peticiones)	Timeout / Circuit Breaker (cortar llamadas lentas o reventar el circuito para proteger el sistema)
Servicios que siempre cambian juntos (servicios muy pequeños pero que dependen de los mismos cambios constantes)	Descomponer por capacidad de negocio (reagrupar o rediseñar los límites del dominio)
Una operación toca datos de varios servicios (transacciones complejas que cruzan múltiples bases de datos)	Saga (secuencia de transacciones locales coordinadas por eventos o pasos compensatorios)
No se sabe por dónde pasó una petición (imposible rastrear errores cuando una llamada salta entre 10 servicios)	Trazas distribuidas (inyectar un identificador único o ID de correlación en cada petición para seguir su ruta)

Patrones y cuando usarlos

Patrón API Gateway

Qué es: Punto de entrada único que recibe las peticiones de los clientes y las enruta al microservicio correcto.

Cuándo usarlo: Con varios clientes (web, móvil) y si quieres ocultar las URLs internas y centralizar autenticación y rate limiting.

Patrón Base de Datos por Servicio

Qué es: Cada microservicio es dueño exclusivo de su base de datos o esquema; nadie comparte tablas.

Cuándo usarlo: Siempre que busques desacoplamiento y despliegue autónomo: si comparten BD, un cambio de esquema rompe ambos.

Patrón Circuit Breaker y Timeouts

Qué es: Detecta fallos o lentitud en un servicio y corta temporalmente la comunicación para evitar fallos en cascada.

Cuándo usarlo: Cuando tus servicios se llaman de forma síncrona (HTTP/gRPC) y una caída puntual no debe tumbar todo.

Patrón Saga

Qué es: Transacción distribuida como secuencia de pasos locales; cada servicio emite un evento, con compensaciones si algo falla.

Cuándo usarlo: Cuando una operación (ej. crear un pedido) toca varias bases de datos y no hay transacciones tradicionales.

Patrón Trazas Distribuidas

Qué es: Un identificador único (Correlation ID) por petición para rastrearla entre los servicios.

Cuándo usarlo: Desde que superas 3 o 4 servicios; sin esto es casi imposible depurar.

Patrones y cuando **no** usarlos

Patrón API Gateway

Sistemas pequeños, apps internas simples o prototipos: los clientes pueden hablar directo con los servicios, sin overhead de enrutamiento.

Patrón Base de Datos por Servicio

Monolito modular o datos muy relacionados: separar las bases solo genera joins distribuidos injustificados.

Patrón Circuit Breaker

Llamadas locales o síncronas simples y confiables: la latencia es despreciable y el fallo es rarísimo.

Patrón Saga

Operaciones dentro de una sola base de datos: una transacción ACID basta y las compensaciones solo suman complejidad.

Patrón Trazas Distribuidas

Arquitecturas de menos de 3 servicios: los logs tradicionales bastan para depurar.

Casos de Aplicación en la Industria

Casos de éxito



Netflix

Cientos de servicios en AWS; creó herramientas que popularizaron el estilo y uso de API Gateway y Circuit Breaker



Amazon

Equipos comunicados solo por interfaces de servicio desde los 2000

mercado libre

Mercado Libre

Miles de microservicios; cerca de la mitad del tráfico en Go

Casos donde se abusó de su uso

Uber

Uber

Miles de servicios; los reagrupó por dominio por exceso de complejidad



Segment

Volvió a un monolito por la sobrecarga operativa

prime video

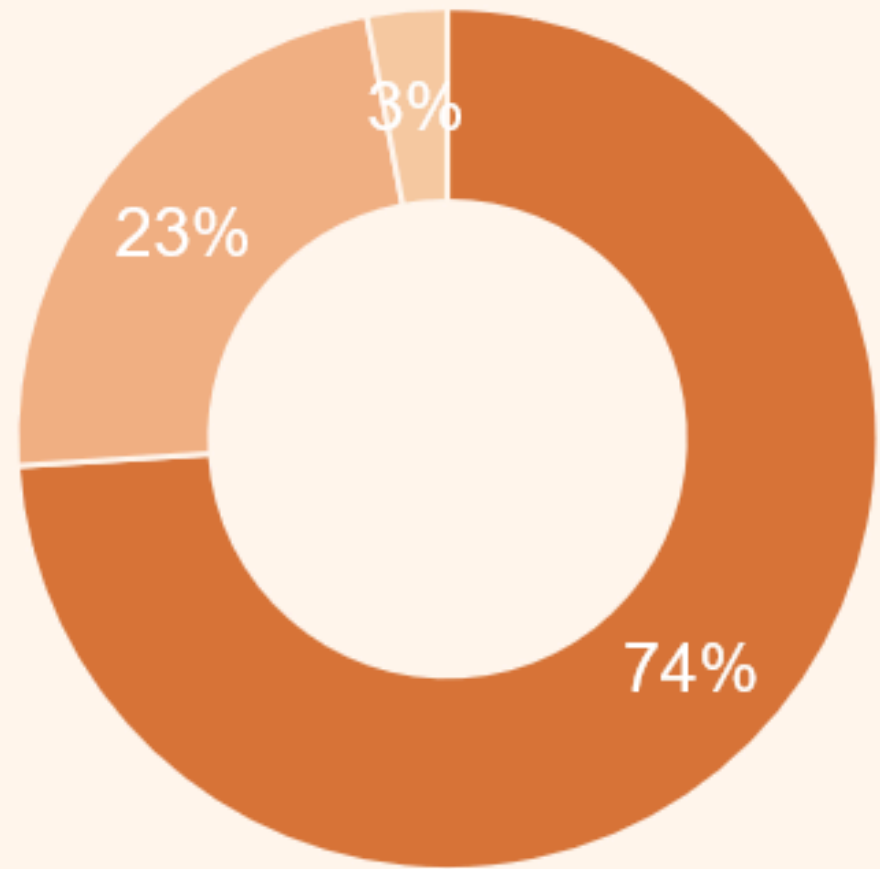
Prime Video

Pasó un sistema a monolito y redujo el costo un 90 %

Uso de microservicios

Qué tan extendidos están en equipos y empresas de tecnología

Organizaciones que los usan



■ Ya los usan ■ Planean adoptarlos ■ Ni usan ni planean

46%

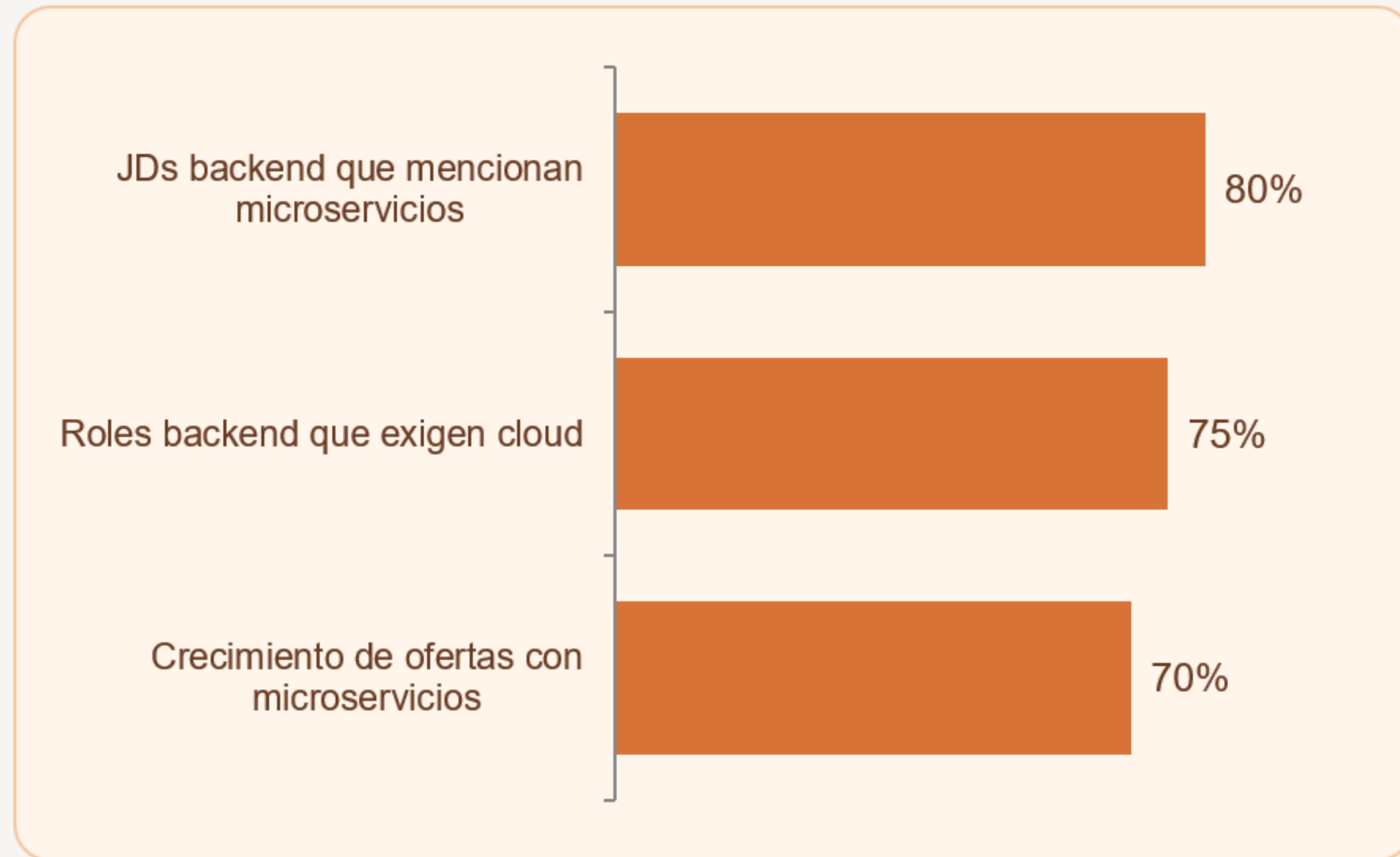
De los desarrolladores backend usan microservicios (CNCF y SlashData, nov. 2025)

77%

De los backend usa al menos una tecnología cloud native; API gateways llega al 50% (CNCF y SlashData, 2025)

Microservicios en las ofertas laborales

Qué tan común es que las vacantes backend los pidan



2,337

Vacantes "backend microservice developer" listadas en Glassdoor EE. UU.

Fuentes: DEV Community (guía backend 2026, "80%+"); Talent500 (2025, ~75%); MoldStud (+70% en un año); Glassdoor (listado de vacantes).

Tecnologías del stack

FRONTEND

Flutter

Un código para Windows y Android; recibe los mensajes como un Stream de Dart

BACKEND

Go

Binario pequeño; miles de conexiones con goroutines y channels

INTEGRACIÓN

gRPC

Contrato .proto que genera Go y Dart; streaming sobre HTTP/2

PERSISTENCIA

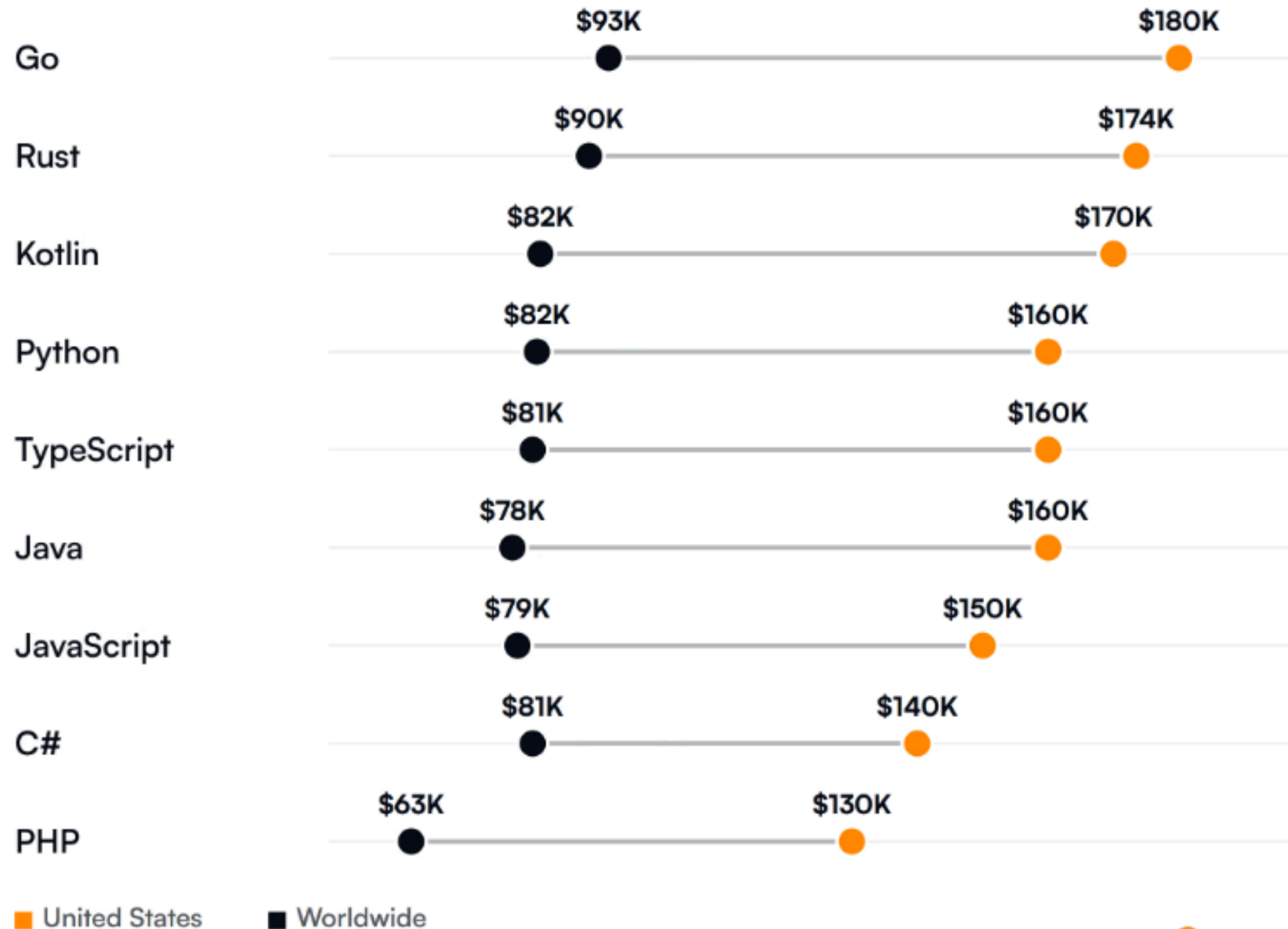
Cassandra

Escrituras masivas y alta disponibilidad, sin nodo maestro

Resumen de datos de salarios- Stack Overflow

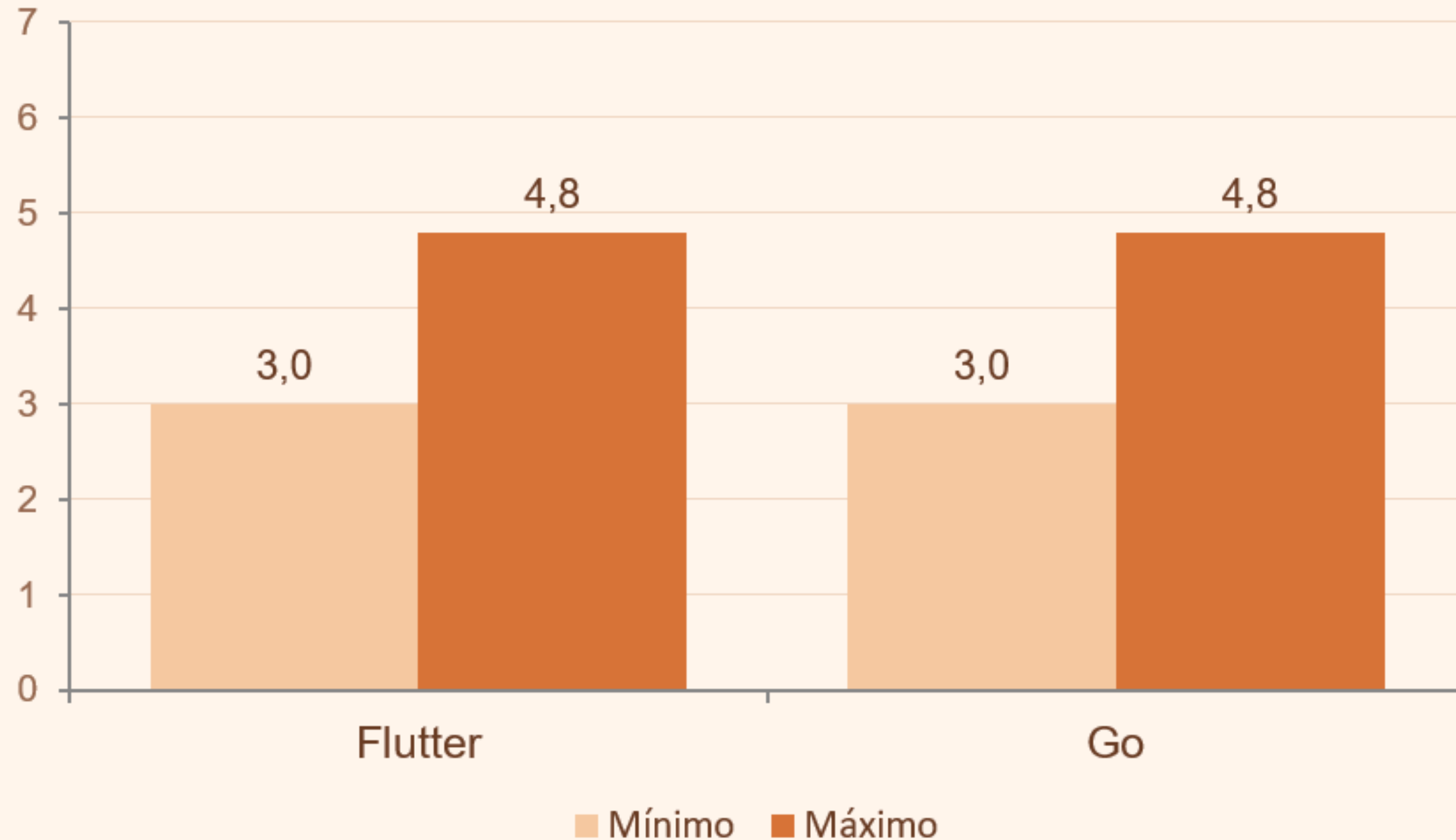
Developer pay by language

Median USD pay, employed professional developers, 2025. Stack Overflow survey data.



Sueldo junior por tecnología

0 a 2 años · rango mensual estimado en Colombia, millones de COP



Go y Flutter

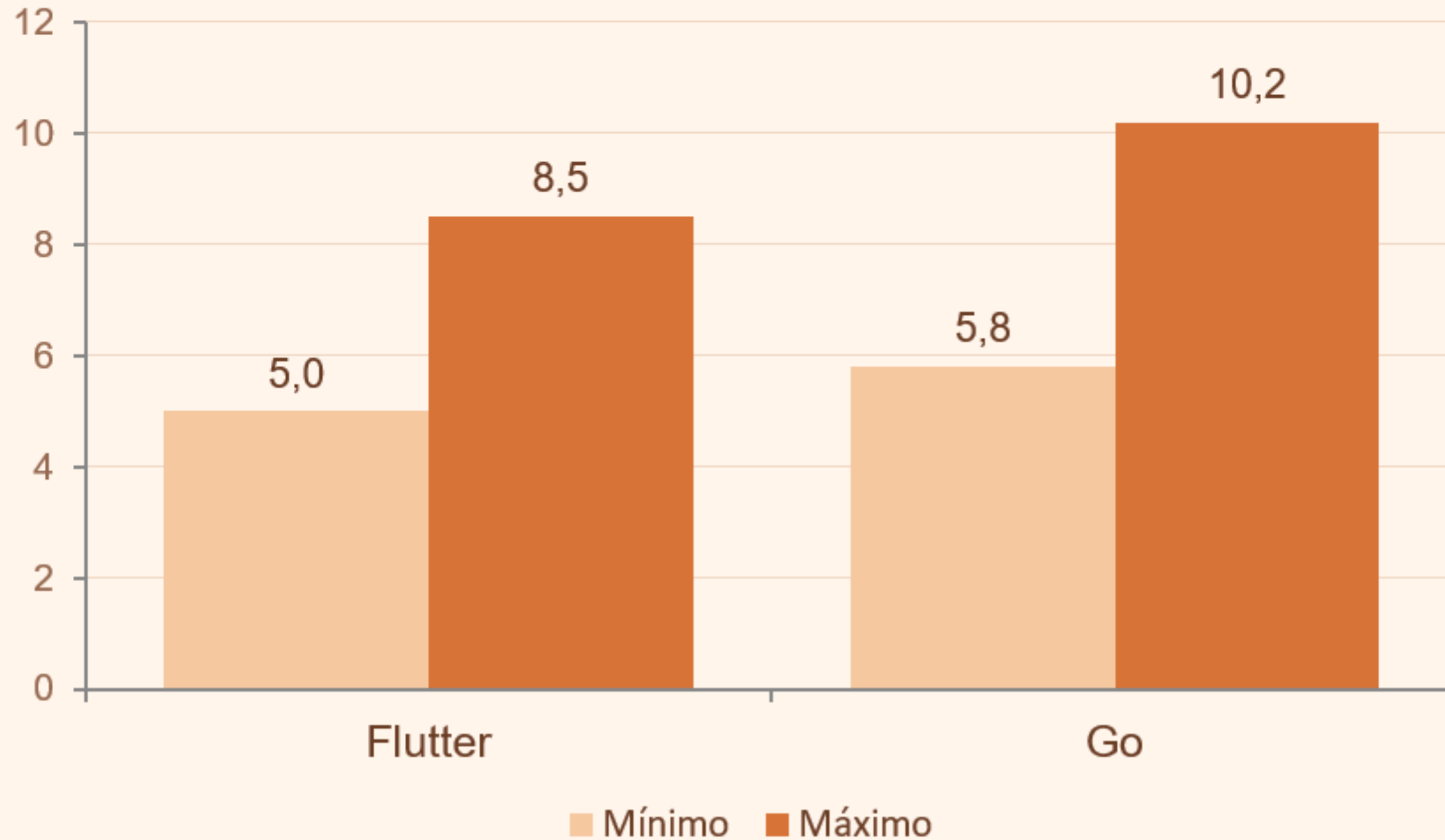
Mismo rango junior: 3 a 4.8 M. La prima de Go aplica desde semi-senior.

Por qué solo dos

gRPC y Cassandra se piden sobre todo en perfiles senior.

Sueldo semi-senior por tecnología

2 a 5 años · rango mensual estimado en Colombia, millones de COP



Go

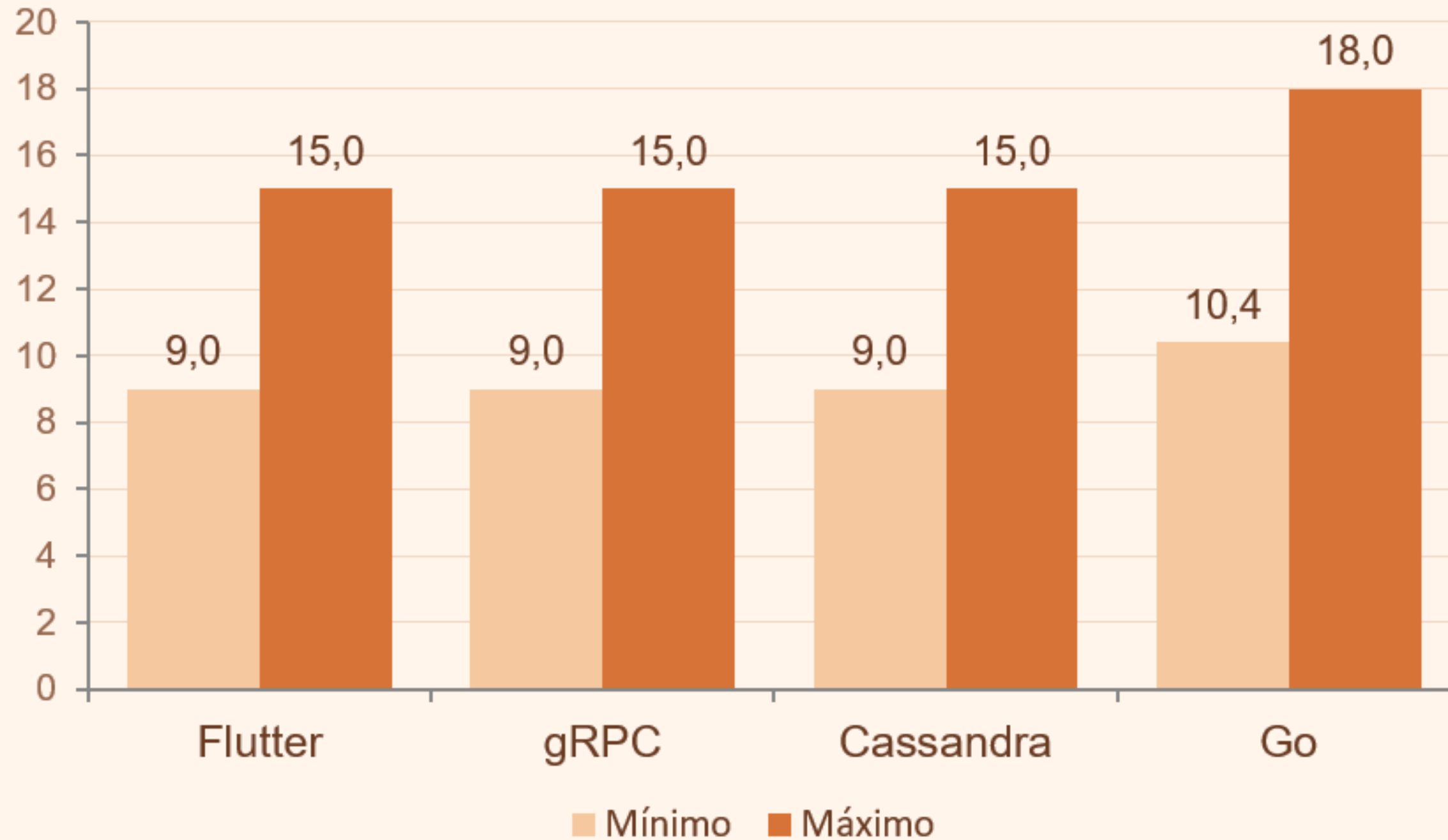
Base semi + 15–20% de prima: 5.8 a 10.2 M.

Flutter

Rango semi estándar: 5 a 8.5 M. Muchas vacantes en este nivel.

Sueldo senior por tecnología

+5 años / arquitectura · rango mensual estimado en Colombia, millones de COP



Go

Base senior + 15–20% de prima: 10.4 a 18 M.

Flutter y gRPC

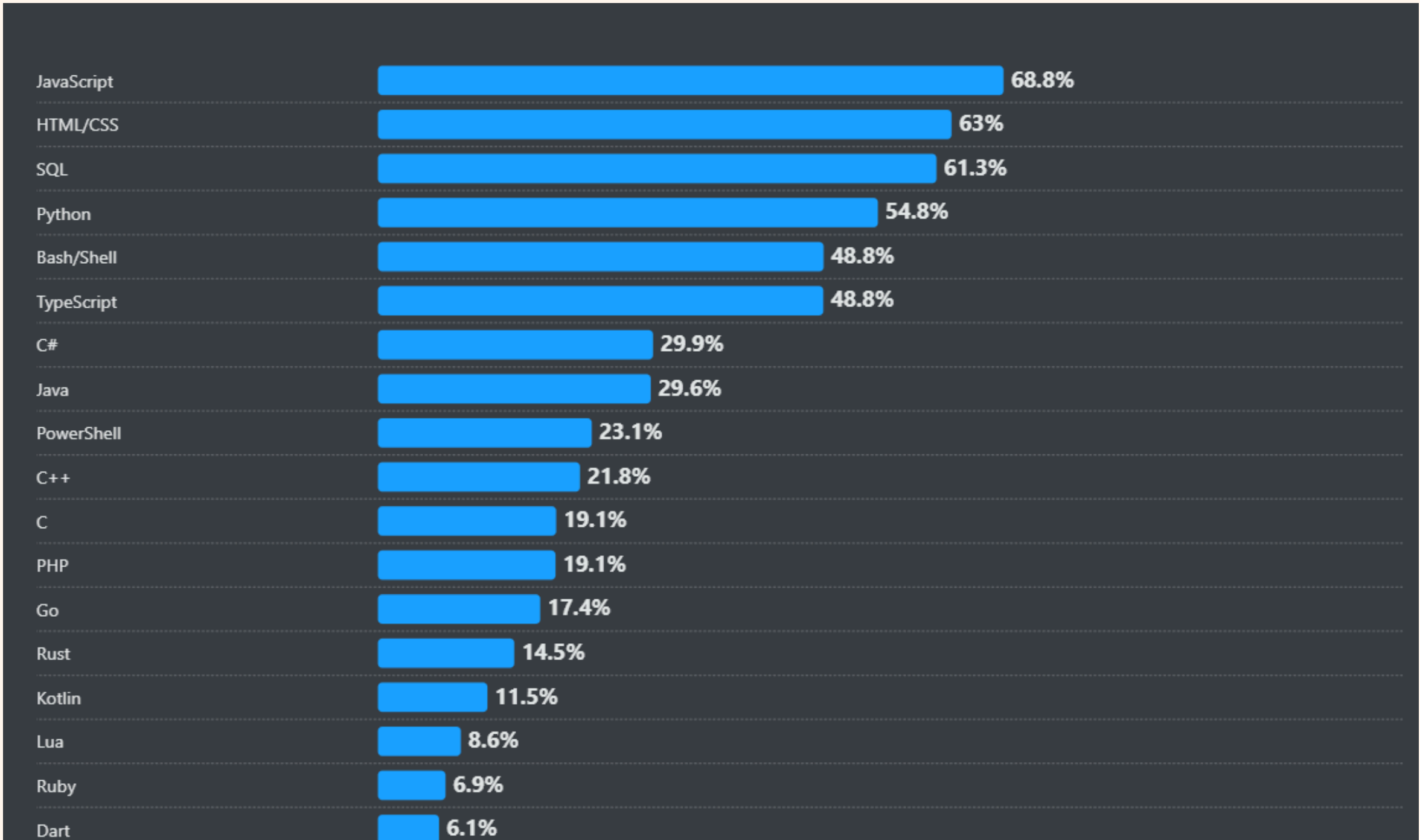
Rango senior estándar: 9 a 15 M.

Cassandra

Piso de 9 M; suele superarlo (senior o arquitecto de datos).

Popularidad de lenguajes

% de desarrolladores que los usaron en el último año



17.4%

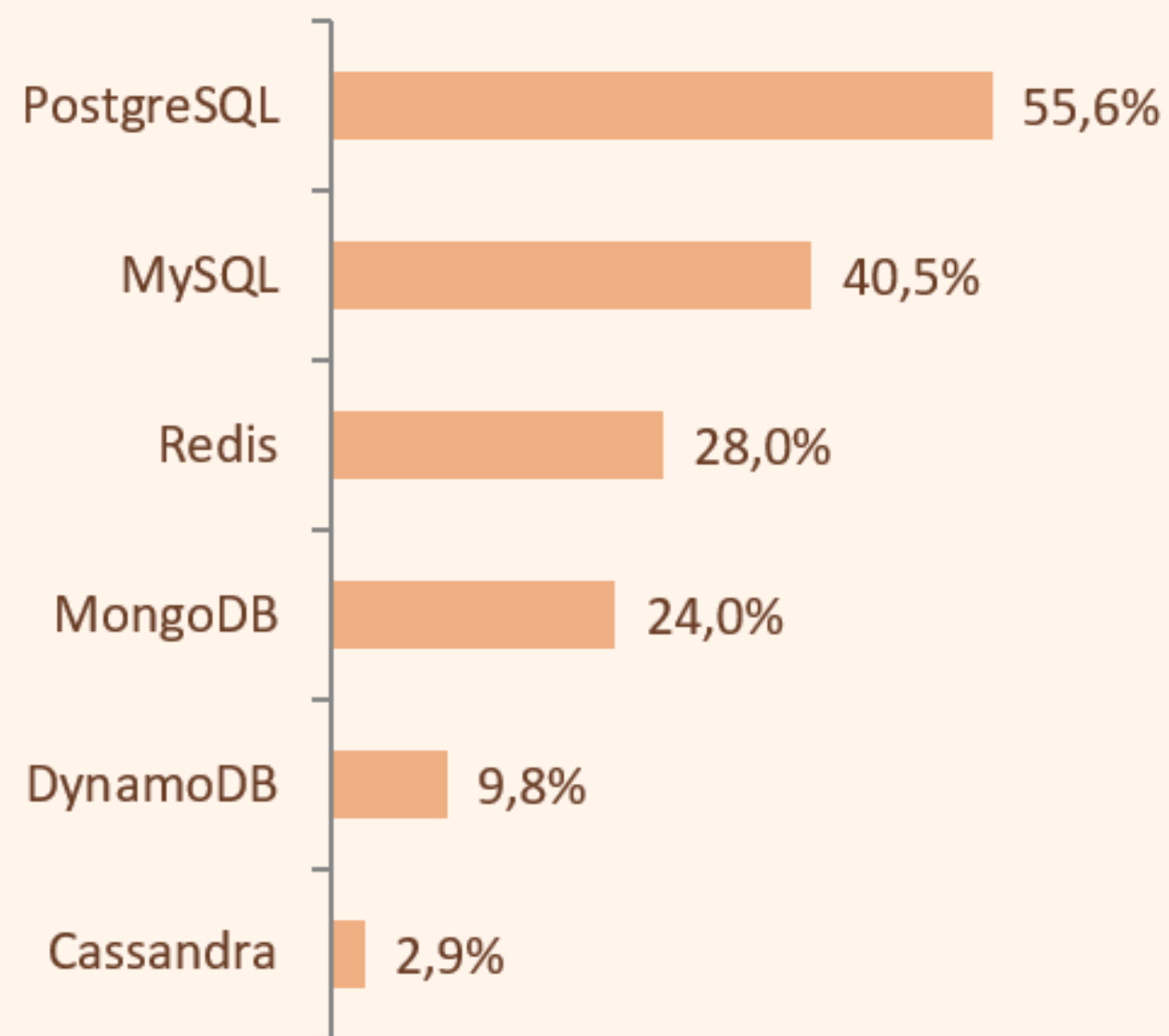
Go: uso en 2025 (+2 pts vs. 2024)

6.1%

Dart, el lenguaje de Flutter

Bases de datos y herramientas

Bases de datos



¿Qué tan común es ver ofertas?

Demanda por tecnología en el mercado

Go

Muy alta

Demanda altísima en fintechs, pasarelas de pago y sistemas distribuidos. Salarios base 15–20% más altos.

Flutter

Alta

Muy pedida por startups y agencias que lanzan Android e iOS con un solo código. Muchas vacantes junior y semi.

gRPC

Media-alta

Requisito o plus en microservicios avanzados; suele pedirse junto con Go o Java en perfiles senior.

Cassandra

Nicho

Nicho de bases NoSQL distribuidas (2.9% de uso). Quien la domina se cotiza en senior y arquitectura de datos.

Flutter: definición, características e historia



Framework de Google para crear apps móviles, web y de escritorio con un solo código, escrito en Dart.

Lo que NO es

- No es un lenguaje (es Dart)
- No usa componentes nativos: dibuja su propia UI
- No es un backend ni una base de datos

Un código, muchas plataformas

Android, iOS, web y escritorio

Hot reload

Cambios visibles en segundos

Todo es un widget

UI declarativa por composición

Motor propio

Skia e Impeller dibujan la UI

HISTORIA Y EVOLUCIÓN

2011

2015

2018

2021

2022

Google presenta Dart

Prototipo "Sky", luego
Flutter

Flutter 1.0 móvil

Flutter 2: web estable

Flutter 3: escritorio
estable

Flutter: ventajas, usos y casos reales



Ventajas

- Un equipo para iOS y Android
- UI idéntica en todo
- Desarrollo rápido

Desventajas

- Apps más pesadas
- Depende de Dart
- Plugins para lo nativo nuevo

CASOS REALES



Google Pay

Pagos



BMW

App My BMW



Alibaba

Xianyu



Nubank

App bancaria

Úsalo cuando

- MVP y startups
- Diseño propio cuidado
- Móvil y web a la vez

Evítalo cuando

- Juegos 3D exigentes
- APIs nativas recientes
- Web con SEO fuerte

Su rol en el stack

Capa cliente: la app que ve el usuario.
Consume los microservicios en Go por API (REST o gRPC).

Go: definición, características e historia



Lenguaje compilado y de tipado estático creado en Google para servicios rápidos y concurrentes.

Lo que NO es

- No es OOP clásico: sin clases ni herencia
- No es un framework: es un lenguaje
- No es ideal para interfaces gráficas

Concurrencia simple

Goroutines y canales

Binario único

Sin dependencias, fácil de desplegar

Recolector de basura

Memoria automática, baja latencia

Biblioteca estándar

HTTP, JSON y testing incluidos

HISTORIA Y EVOLUCIÓN

2007

2009

2012

2018

2022

Se diseña en Google

Código abierto

Go 1.0 y compatibilidad

Go 1.11: módulos

Go 1.18: genéricos

Go: ventajas, usos y casos reales

 **GO** Go

Ventajas

- Rápido y liviano
- Compila y despliega simple
- Concurrencia nativa

Desventajas

- Errores repetitivos
- Menos abstracciones
- Poco fuerte en UI y datos

Úsalo cuando

- APIs y microservicios
- Herramientas de nube y CLI
- Muchas conexiones a la vez

Evítalo cuando

- Apps con UI
- Machine learning
- Lógica muy orientada a objetos

CASOS REALES



Docker

Contenedores



Kubernetes

Orquestador

Uber

Uber

Servicios internos



Twitch

Chat y streaming

Su rol en el stack

Motor de los microservicios: implementa la lógica de negocio y expone sus servicios por gRPC.

Cassandra: definición, características e historia



Base de datos NoSQL distribuida de columnas anchas, sin nodo maestro, para gran escala y alta disponibilidad.

Lo que NO es

- No es relacional: sin JOINS ni ACID completo
- No sirve para consultas ad hoc
- No es una base en memoria ni un buscador

Sin punto único de falla

Arquitectura peer-to-peer

Escala en horizontal

Sumas nodos sin detener el servicio

Consistencia ajustable

Nivel elegido por consulta

Escrituras muy rápidas

Commit log y memtable (LSM)

HISTORIA Y EVOLUCIÓN

2008

2009

2010

2021

2024

Facebook la crea
(Dynamo + Bigtable)

Incubadora de Apache

Proyecto Apache top-level

Cassandra 4.0

Cassandra 5.0: SAI y vectores

Cassandra: ventajas, usos y casos reales



Ventajas

- Altísima disponibilidad
- Escala lineal
- Escrituras masivas

Desventajas

- Modelar por consulta cuesta
- Sin JOINS
- Operación exigente

CASOS REALES



Netflix

Datos de usuarios



Apple

Gran escala



Instagram

Datos de actividad



Discord

Migró a ScyllaDB (2023)

Úsalo cuando

- Series de tiempo y logs
- Muchas escrituras y alta disp.
- Varias regiones

Evítalo cuando

- Datos muy relacionales
- Volúmenes pequeños
- Analítica cambiante

Su rol en el stack

Persistencia de los microservicios: cada servicio guarda sus datos de forma distribuida y siempre disponible.

gRPC: definición, características e historia

gRPC

Framework RPC de código abierto que usa HTTP/2 y Protocol Buffers para comunicar servicios.

Lo que NO es

- No es un lenguaje ni una base de datos
- No es REST: define métodos en un .proto
- No corre directo en navegador (gRPC-Web)

Contrato en .proto

Genera cliente y servidor

Protocol Buffers

Formato binario compacto

HTTP/2

Multiplexado y streaming

Multilenguaje

Go, Java, Python, Dart, C++

HISTORIA Y EVOLUCIÓN

2001

2008

2015

2017

2018

Stubby, uso interno en Google

Protobuf pasa a código abierto

Google libera gRPC

Entra a la CNCF

gRPC-Web disponible

gRPC: ventajas, usos y casos reales

gRPC

Ventajas

- Más rápido que JSON
- Contratos estrictos
- Streaming en tiempo real

Desventajas

- Binario, difícil de depurar
- Navegador limitado
- Herramientas extra

CASOS REALES



Google

Origen (Stubby)



Netflix

Entre servicios



Dropbox

Servicios internos



etcd

Base de Kubernetes

Úsalo cuando

- Comunicación entre microservicios
- Streaming y baja latencia
- Equipos multilenguaje

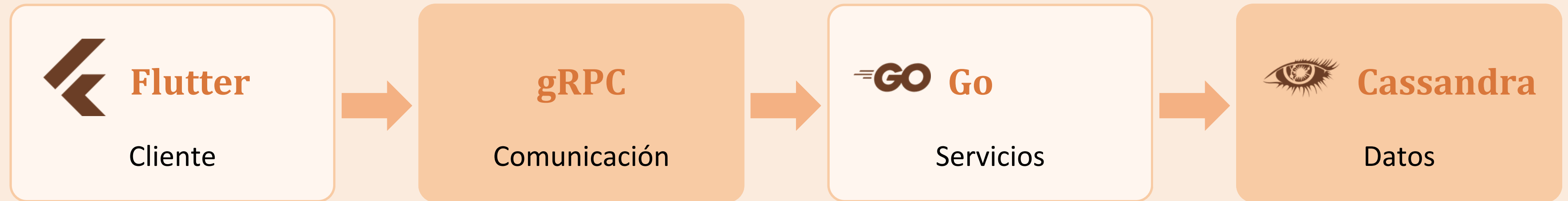
Evítalo cuando

- APIs públicas (REST es más simple)
- Web sin proxy
- Prototipos rápidos

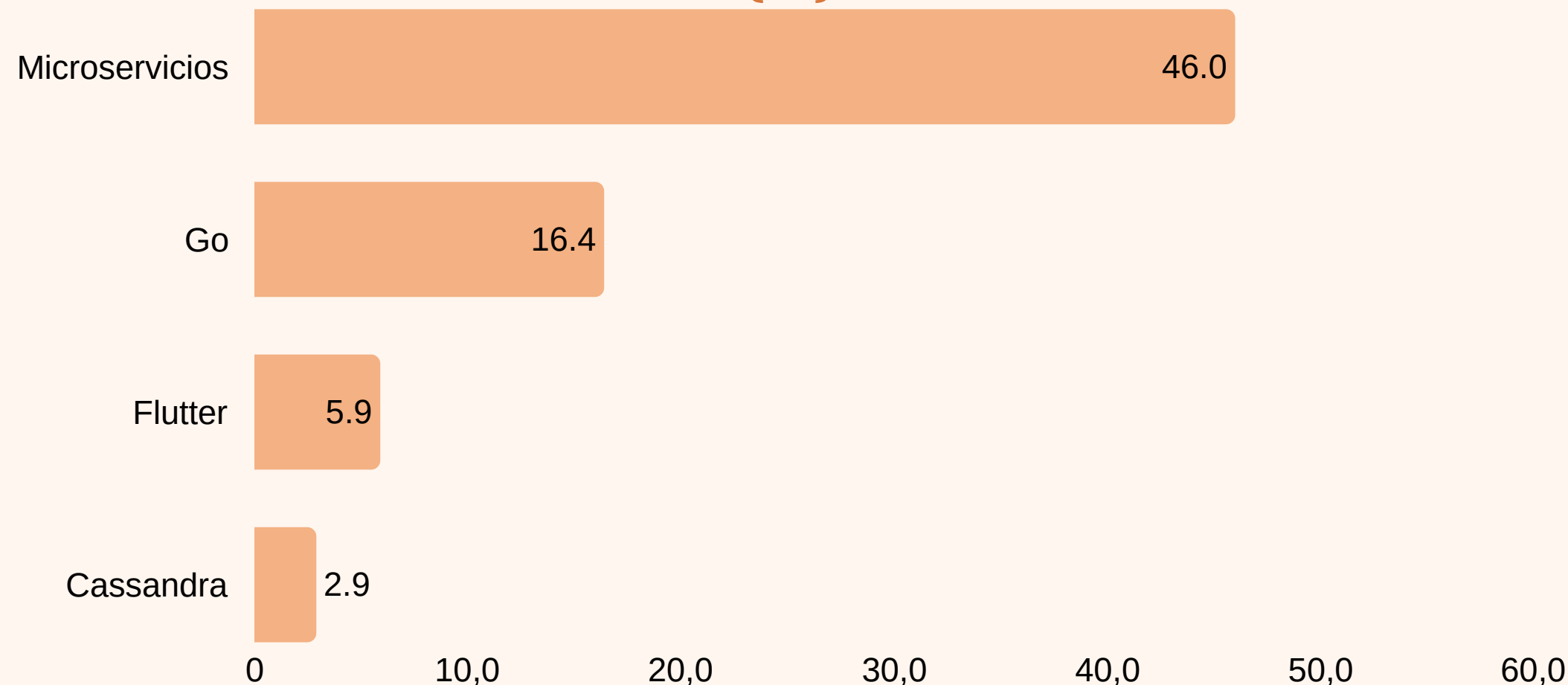
Su rol en el stack

Pegamento entre servicios: los microservicios en Go se hablan por gRPC. Flutter usa grpc-dart o un gateway.

Cómo encajan y qué tan común es el stack



Uso entre desarrolladores (%)



Estilo: microservicios

Servicios pequeños e independientes, cada uno con su base de datos, comunicados por red.

¿Qué tan común es?

Cada pieza es común por separado (Go y gRPC suelen ir juntos). Las cuatro juntas son un nicho; Cassandra es la menos frecuente.

LA PROPUESTA

Un chat por canales



A escribe en un canal y **B lo recibe en vivo.**

Fuera de alcance: login real, búsqueda, mensajes no leídos y notificaciones push.

Atributos de calidad vs estilo (ISO/IEC 25010:2023)

S = soporta · L = limita · N = neutro

Atributo	Estilo (microservicios)	Flutter	Go	Cassandra	gRPC
1. Idoneidad funcional	S: un servicio por capacidad de negocio · L: coherencia entre servicios	S: mismas funciones en todas las plataformas	S: lógica clara y tipada	L: sin JOINS ni ACID completo	S: contratos .proto exactos
2. Eficiencia de desempeño	S: escala por servicio · L: latencia de red	S: compilado, fluido · L: app más pesada	S: rápido, poca memoria	S: escrituras masivas	S: binario sobre HTTP/2
3. Compatibilidad	S: poliglota · L: versionar contratos	S: Android, iOS, web, escritorio	S: binarios multiplataforma	L: CQL propio, no SQL estándar	S: multilenguaje · L: navegador solo con gRPC-Web
4. Capacidad de interacción	N: depende del cliente	S: UI consistente y personalizable	N: no tiene UI	N: no tiene UI	L: depuración binaria · S: código generado
5. Fiabilidad	S: aislamiento de fallos · L: fallos en cascada	L: depende de la red, offline propio	S: errores explícitos	S: sin punto único de falla, replicación	S: deadlines · L: requiere política de reintentos
6. Seguridad	S: aislamiento · L: más superficie de ataque	L: el cliente no es confiable	S: TLS en biblioteca estándar	S: roles y cifrado · L: se configura a mano	S: TLS y mTLS nativos
7. Mantenibilidad	S: equipos y despliegues independientes · L: muchos servicios	S: un código base · L: dependencia de plugins	S: sintaxis pequeña, gofmt	L: cambios de esquema y operación costosos	S: evolución compatible de Protobuf
8. Flexibilidad	S: cambiar tecnología por servicio	S: un código, varias plataformas	S: compilación cruzada	S: on-premise, nube, Kubernetes · L: migrar datos cuesta	S: independiente del lenguaje
9. Safety (seguridad operacional)	N: solo aplica en sistemas críticos	N: no certificado para eso	N: no certificado para eso	N: replicación evita pérdida de datos	N: no aplica

Principios vs estilo

Cómo el estilo y cada tecnología cumplen o tensionan los principios

Principio	Estilo	Flutter	Go	Cassandra	gRPC
SOLID	S: un servicio, una responsabilidad · D: dependen de contratos	Widgets pequeños de una sola tarea	Interfaces implícitas facilitan I y D	Una tabla por consulta (S)	Servicios pequeños por .proto (I)
KISS	L: más complejidad operativa	S: widgets simples	S: lenguaje pequeño	L: modelado y operación complejos	L: más piezas que REST
DRY	L: lógica repetida entre servicios	S: un código, widgets reutilizables	S: genéricos desde Go 1.18 · L: errores repetitivos	L: denormalizar duplica datos por diseño	S: un .proto genera clientes y servidores
YAGNI	Riesgo: adoptarlo sin necesitarlo	Evitar plugins sin uso	S: pocas dependencias	Riesgo: sobredimensionar para poco volumen	Riesgo: usarlo donde REST basta
PoLA (mínimo asombro)	S: APIs consistentes	S: widgets predecibles	S: pocas formas de hacer algo	L: consistencia eventual y tombstones sorprenden	S: contrato explícito
Ley de Demeter	S: hablan solo con sus vecinos por API	Depende: evitar cadenas largas de objetos	S: encapsulación por paquetes	S: cada servicio accede solo a sus tablas	S: expone solo métodos del contrato
STUPID (anti-patrones)	S: bajo acoplamiento · L: acoplamiento por llamadas síncronas	Riesgo: singletons y estado global	Riesgo: globales · S: interfaces para pruebas	Riesgo: optimización prematura y duplicación	Riesgo: acoplarse al .proto
Composición sobre herencia	S: se compone con servicios	S: todo se compone de widgets	S: sin herencia, usa embedding	N: claves compuestas y tipos definidos (UDT)	N: mensajes se componen, sin herencia

ADR: decisiones del stack y costo aceptado

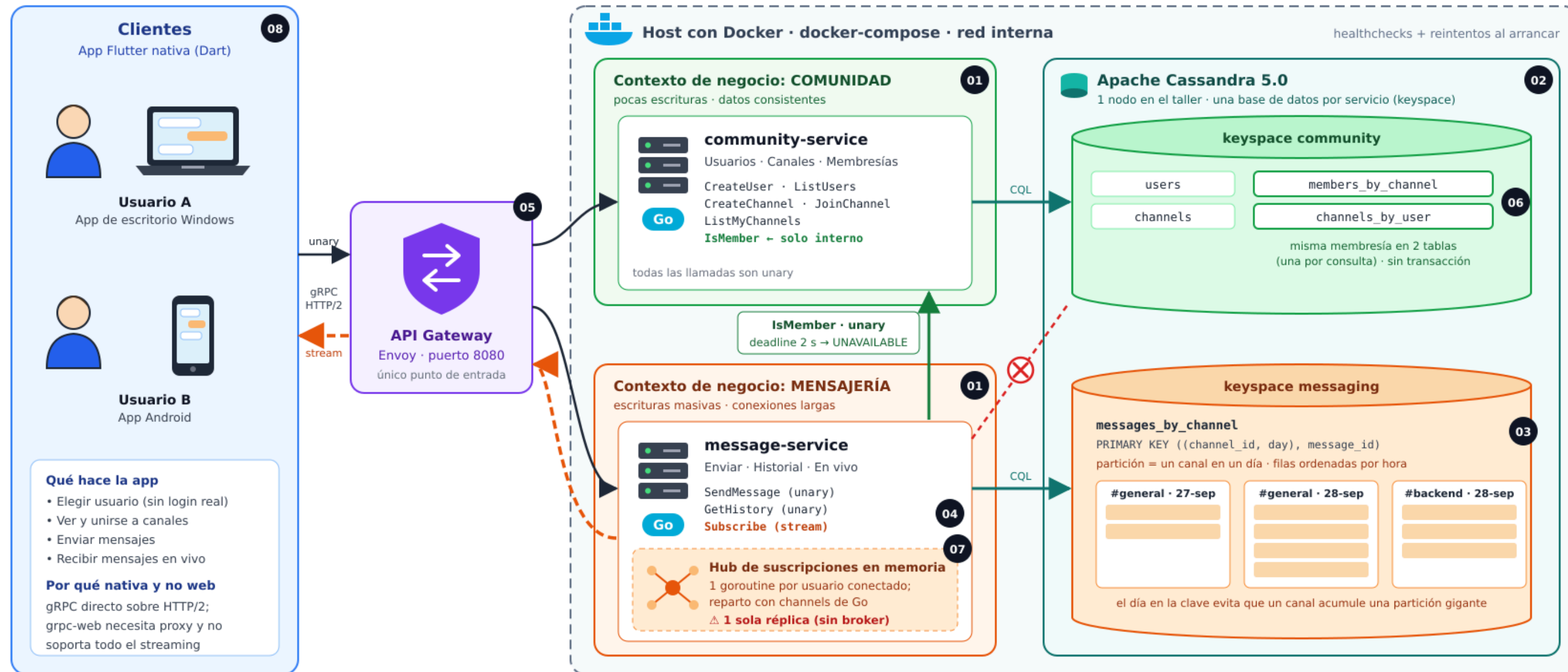
Cada decisión se justifica porque su costo es conocido y aceptado (Flutter · gRPC · Go · Cassandra, en microservicios)

#	Decisión	Afecta a	Costo aceptado
01	Separar por contexto de negocio	Estilo	Una llamada de red para validar membresía
02	Un keyspace por servicio	Cassandra	Sin consultas entre servicios
03	Partición (canal, día)	Cassandra	Varios días = varias particiones
04	Unary + server streaming	gRPC	Sin streaming bidireccional
05	API Gateway con Envoy	gRPC / Flutter	Un componente y un salto más
06	Membresía en 2 tablas (BATCH logged)	Cassandra	Sin aislamiento por un instante
07	Suscripciones en memoria, 1 réplica	Go	Pierde escalabilidad; broker a futuro
08	Cliente nativo, no web	Flutter	La app no corre en navegador

ARQUITECTURA de alto Nivel (HDL)

Chat por canales — Arquitectura de alto nivel (HLD)

Microservicios · Flutter · Go · gRPC · Cassandra · Docker — los círculos negros numerados remiten a los ADRs (ver leyenda)



Cómo se comunican

- gRPC unary: pregunta → respuesta
- - - → gRPC server streaming: el servidor empuja mensajes
- gRPC entre servicios (interno)
- Acceso a datos con CQL (driver gocql)
- - - X - - - Prohibido: un servicio no lee la base de otro

Contratos: un mismo archivo .proto genera el código de Go (servicios) y de Dart (app).

Decisiones de arquitectura (ADRs) visibles en el diagrama

- 01 Servicios separados por contexto de negocio, no uno por entidad
- 02 Una base de datos (keyspace) por servicio; nadie lee la base de otro
- 03 Mensajes particionados por (canal, día) para repartir carga y acotar tamaño
- 04 Enviar con unary + recibir con server streaming (no bidireccional)
- 05 API Gateway (Envoy) como único punto de entrada
- 06 Membresía duplicada en 2 tablas, sin transacción
- 07 Suscripciones en memoria: 1 réplica; broker a futuro
- 08 Cliente Flutter nativo (Windows/Android), no web

Flujo principal: A envía → Envoy → message-service → IsMember en community-service → guarda en messaging.messages_by_channel → el hub lo reparte → B lo recibe por Subscribe

Diagrama de Contexto

Diagrama de Contexto (C4 · Nivel 1) — Chat por canales

Quién usa el sistema y con qué otros sistemas se relaciona. Todavía no se ve cómo está construido por dentro.



Leyenda (notación C4)

Persona

Sistema de software (el nuestro)

Sistema externo fuera del alcance del taller

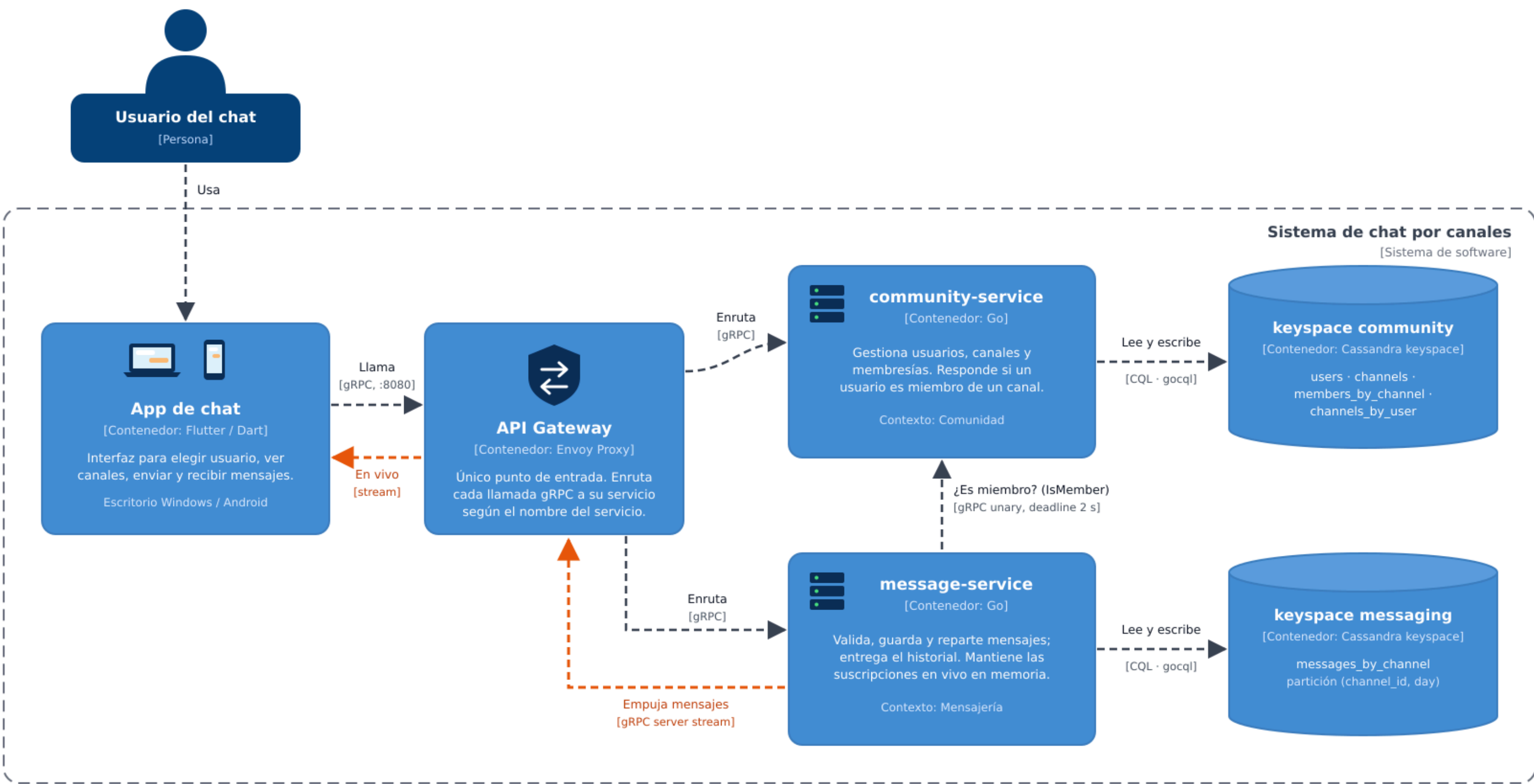
Relación [tecnología]

El sistema no depende de ningún sistema externo en esta versión: todo corre en contenedores propios. Los dos externos en gris muestran qué se integraría en un producto real y por eso quedan declarados como fuera de alcance.

Diagrama Contenedores

Diagrama de Contenedores (C4 · Nivel 2) — Chat por canales

Las piezas que se despliegan por separado, su tecnología y cómo se comunican.



Leyenda (notación C4)

- Persona
- Contenedor (se despliega por separado)
- Almacén de datos
- Petición [tecnología]
- Mensajes en vivo (server streaming)

Cada servicio solo accede a su propio keyspace. Ambos keyspaces viven en el mismo nodo de Cassandra (ver diagrama de Despliegue).

FLUJO PRINCIPAL: Enviar un mensaje, paso a paso

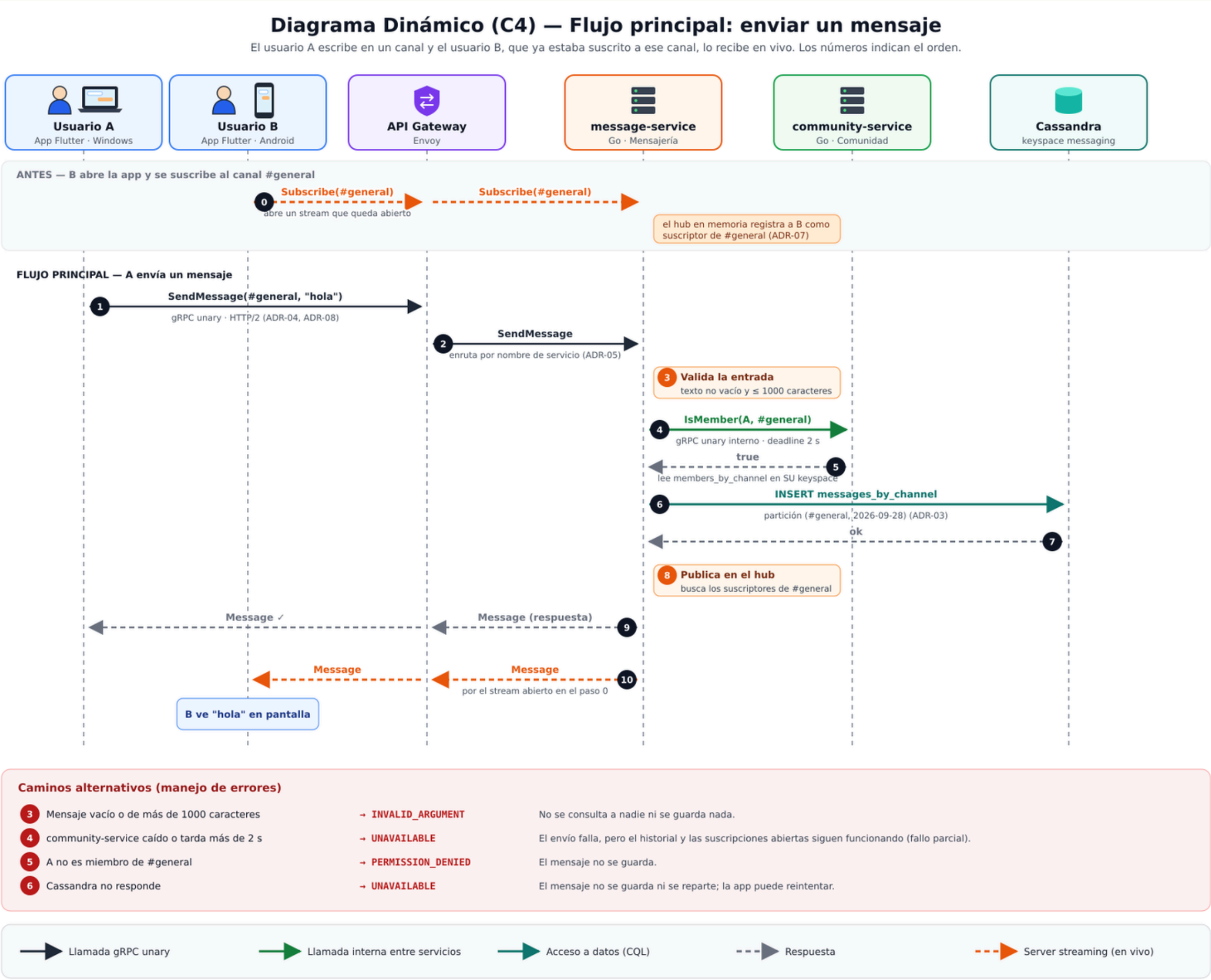
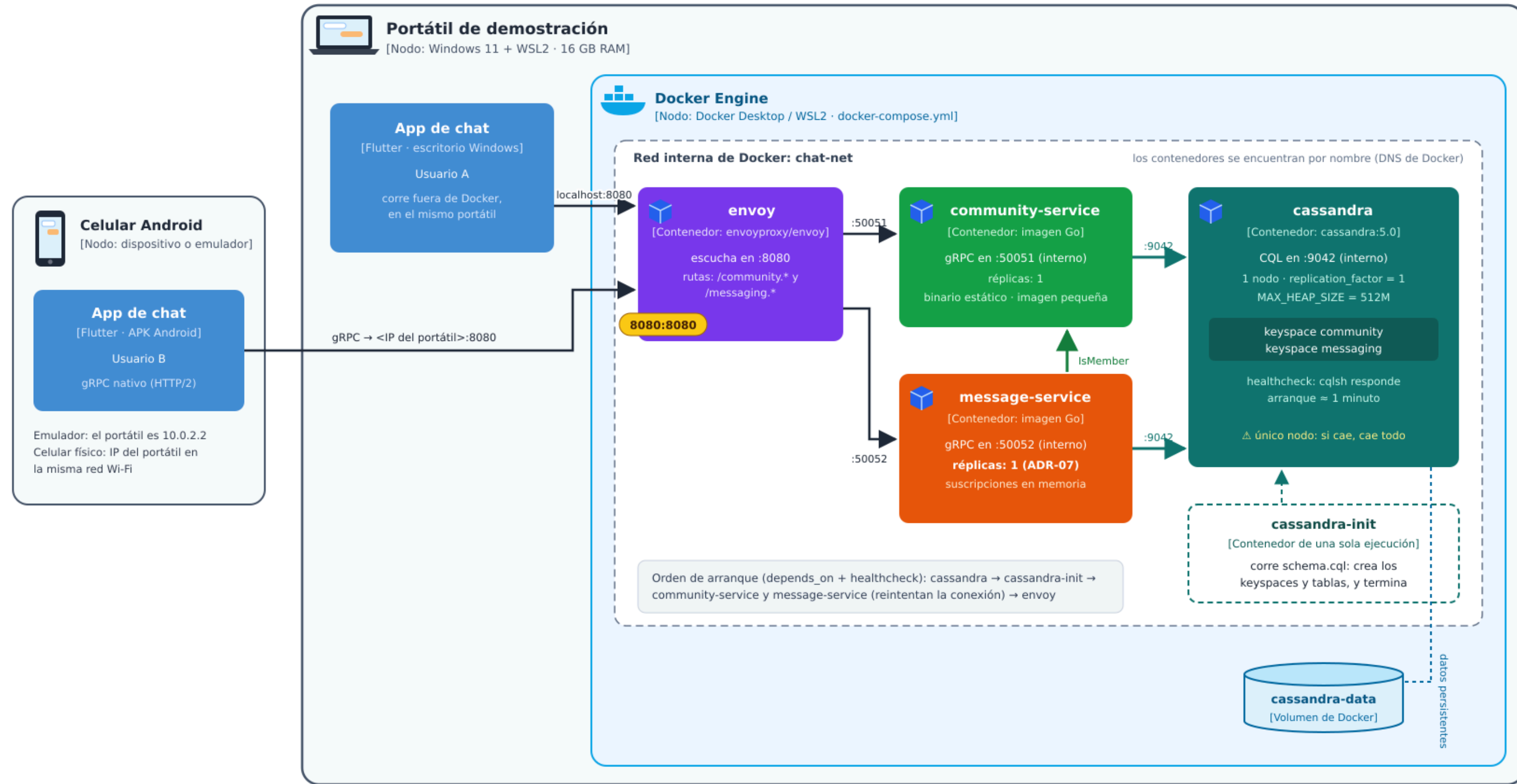


Diagrama de Despliegue

Diagrama de Despliegue — Chat por canales (entorno de demostración)

Dónde corre cada contenedor, qué puertos expone y cómo llegan los clientes. Todo se levanta con un solo docker compose up.



Leyenda

Nodo (máquina o entorno) App cliente Contenedor de Docker Único puerto publicado hacia afuera

Solo Envoy es accesible desde fuera de Docker. Los servicios y Cassandra solo existen dentro de la red chat-net. Los puertos :50051 y :50052 son la propuesta del equipo; si se cambian, se actualiza aquí.

Diagrama de Componentes

Diagrama de Componentes (C4 · Nivel 3) — message-service

Las piezas internas del servicio de mensajería (paquetes de Go) y cómo colaboran. Es el servicio con más lógica del sistema.

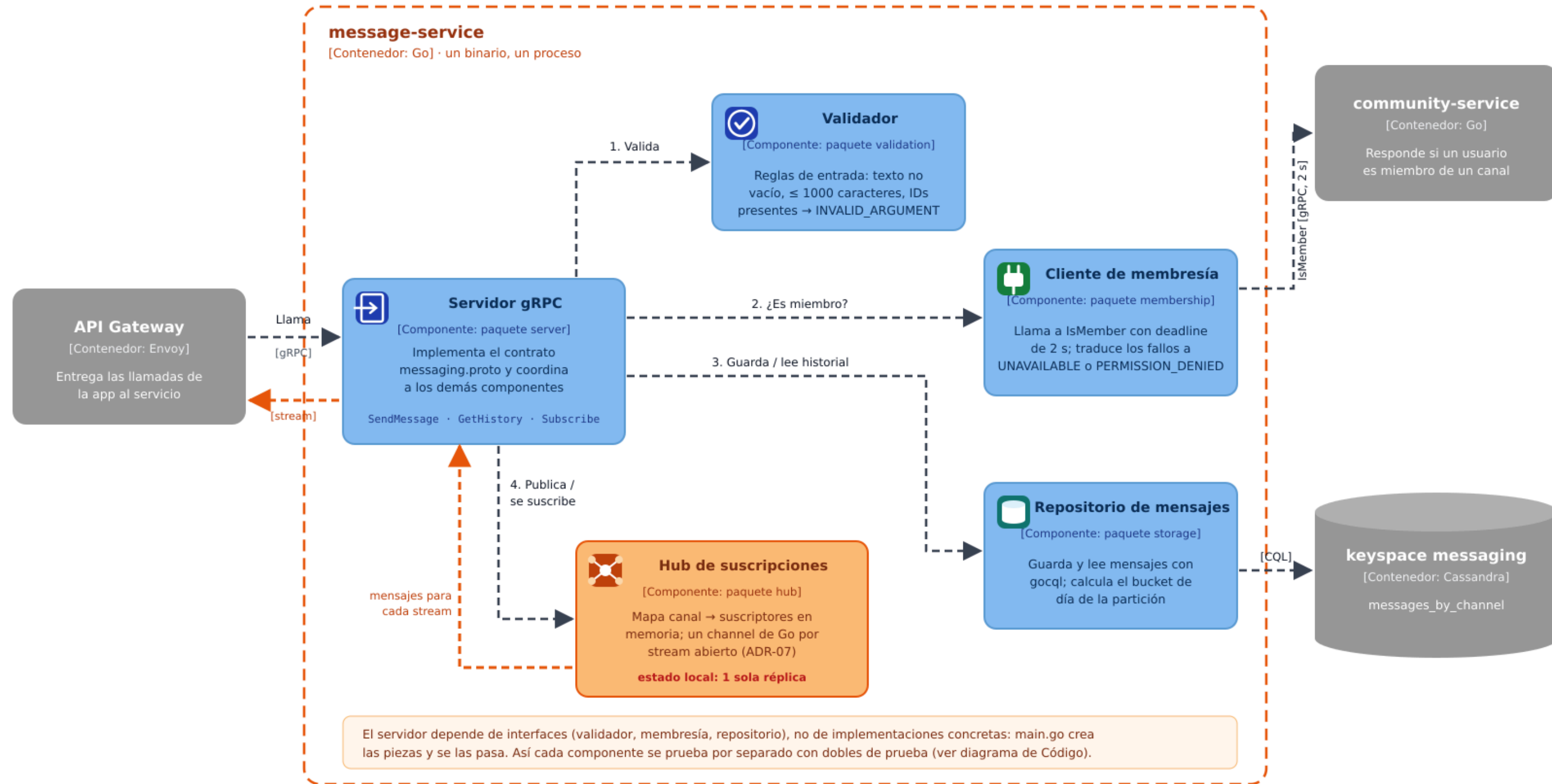
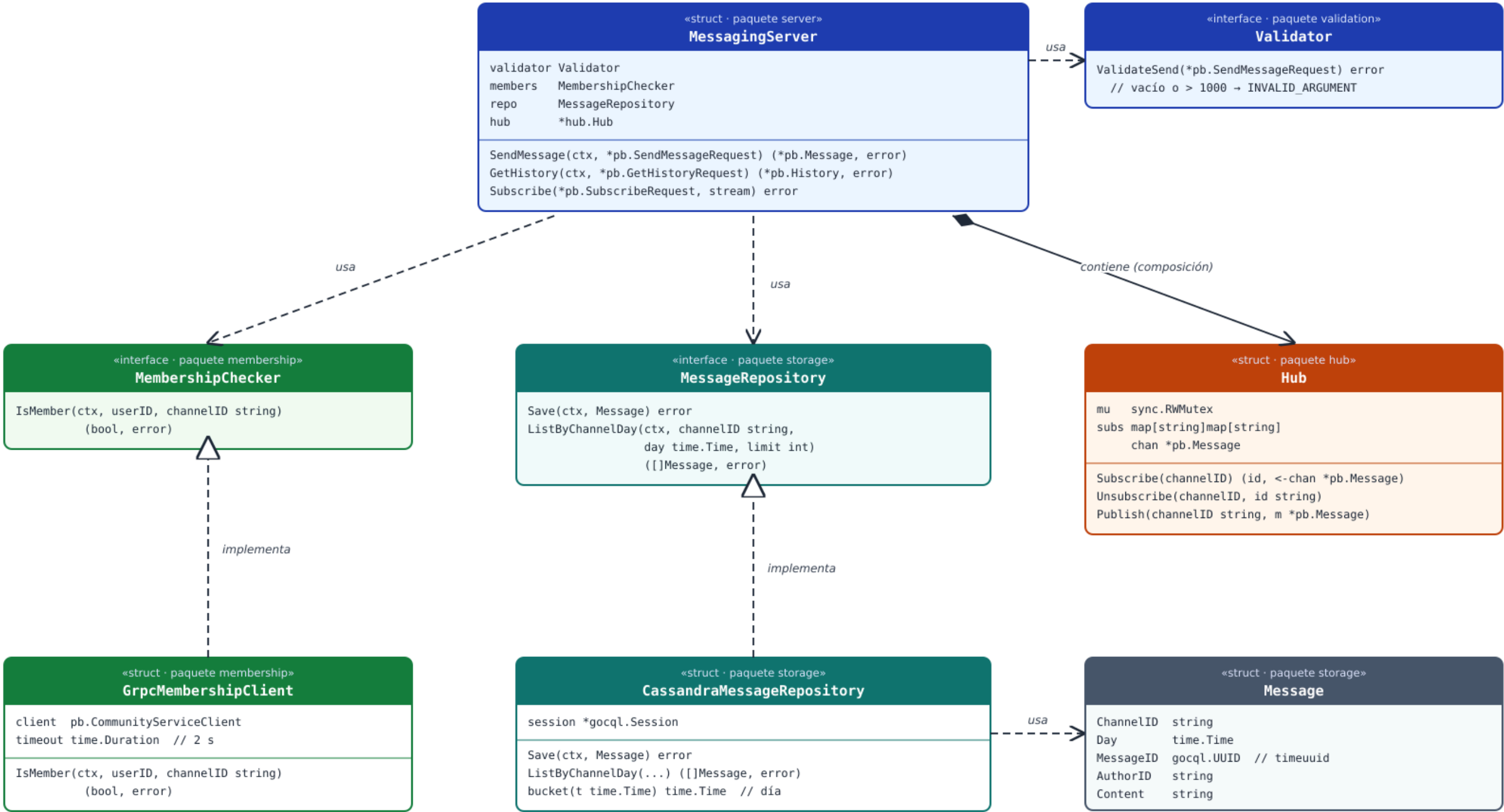


Diagrama de Código: Interfaces y composición en Go

Diagrama de Código (C4 · Nivel 4) — message-service en Go

Tipos principales del servicio. Go no tiene clases ni herencia: usa structs, interfaces implícitas y composición.

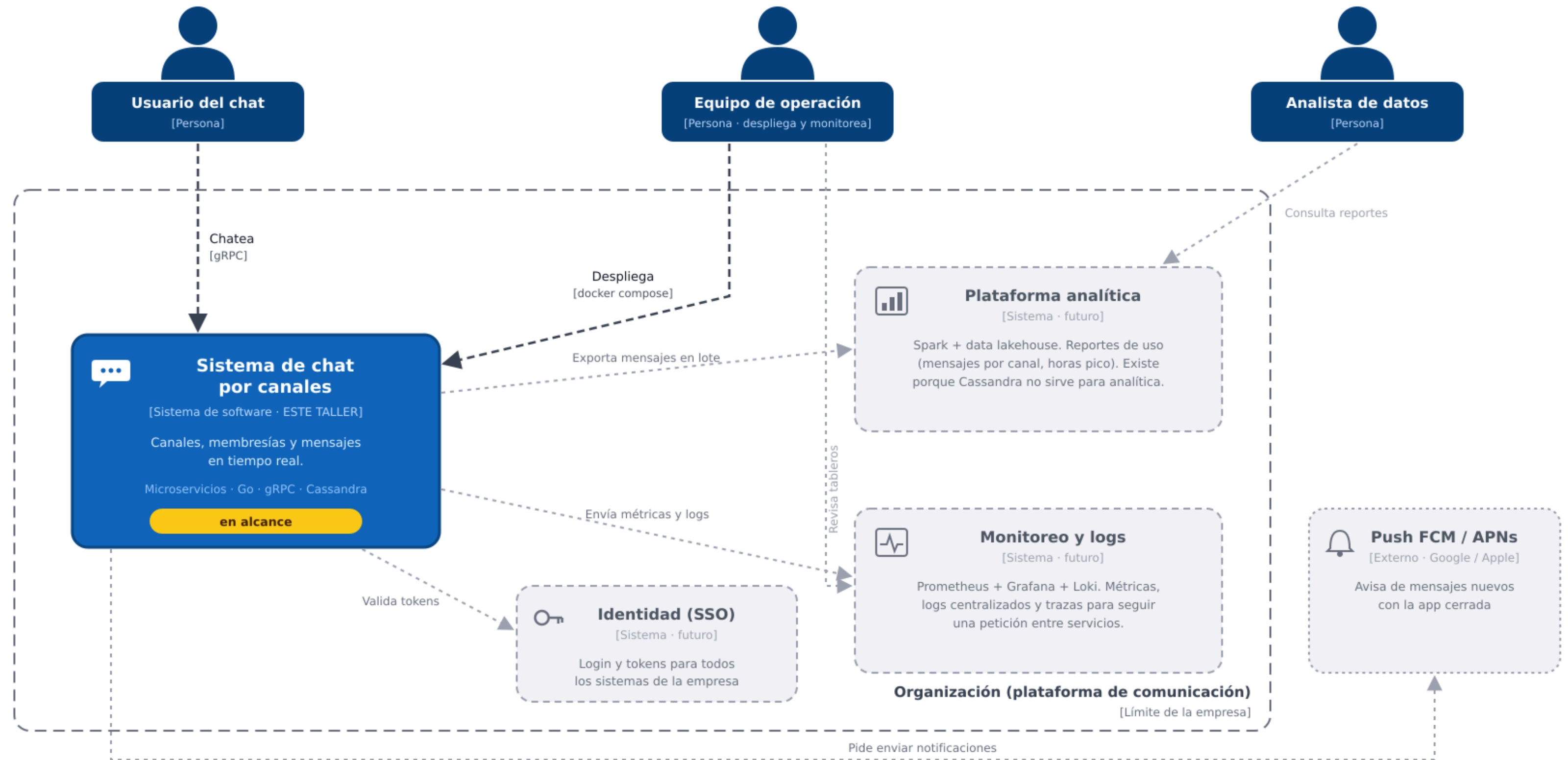


◆ Composición: MessagingServer contiene un Hub. - - ▷ Implementa: el struct cumple la interfaz sin declararlo (interfaces implícitas de Go). - - > Usa: depende de ese tipo.
El servidor depende de interfaces (MembershipChecker, MessageRepository, Validator), no de gRPC ni de Cassandra: es la D de SOLID. En las pruebas se reemplazan por dobles en memoria. main.go crea GrpcMembershipClient, CassandraMessageRepository y Hub, y se los pasa a MessagingServer (inyección de dependencias).

Diagrama System landscape

System Landscape — El chat dentro de su ecosistema

Todos los sistemas y personas alrededor del chat en una organización real. En el taller solo existe el sistema azul; el resto es la evolución prevista.



Sistema construido en el taller

Sistema previsto (futuro)

Persona

Relación que existe hoy

Relación futura

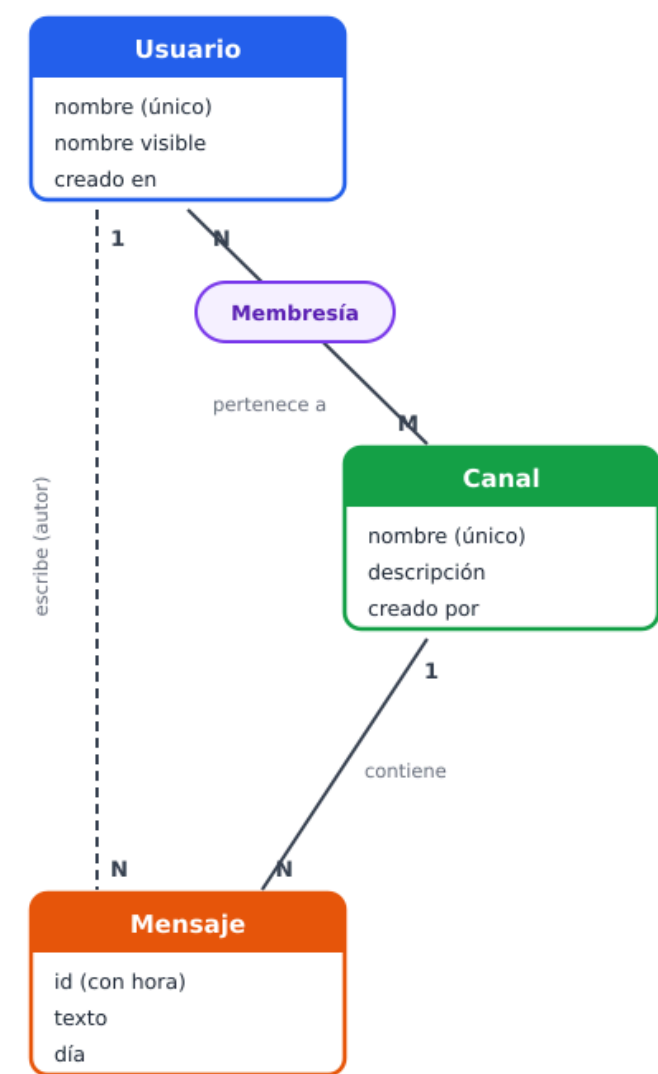
Cada sistema futuro responde a una limitación declarada del taller: analítica (Cassandra), trazas (analizabilidad), identidad (sin login real), push (app cerrada).

Modelo de Datos

Modelo de datos — de las entidades a las tablas de Cassandra

En Cassandra se modela para leer: primero las entidades, después las consultas de la app y, al final, una tabla por consulta.

1 Modelo conceptual



3 entidades de negocio interrelacionadas.
Usuario y Canal: relación N:M (membresía).
Canal y Mensaje: 1:N. Usuario y Mensaje: 1:N.

2 Consultas de la app

Q1
Crear usuario sin repetir nombre
INSERT ... IF NOT EXISTS
→ users

Q2
Listar usuarios (elegir con quién entrar)
SELECT * FROM users
→ users

Q3
Crear canal sin repetir nombre
INSERT ... IF NOT EXISTS
→ channels

Q4
Unirse a un canal
2 INSERT (ver nota)
→ members_by_channel + channels_by_user

Q5
¿El usuario es miembro? (IsMember)
WHERE channel_id=? AND user_id=?
→ members_by_channel

Q6
Mis canales
WHERE user_id=?
→ channels_by_user

Q7
Enviar mensaje
INSERT ... (canal, día, now())
→ messages_by_channel

Q8
Historial del día (los más nuevos primero)
WHERE channel_id=? AND day=? LIMIT 50
→ messages_by_channel

3 Tablas (una por consulta)

keyspace community
solo lo usa community-service

users	Q1 · Q2
user_id	text K
display_name	text
created_at	timestamp

channels	Q3
channel_id	text K
description	text
created_by	text
created_at	timestamp

members_by_channel	Q4 · Q5
channel_id	text K
user_id	text C↑
joined_at	timestamp

channels_by_user	Q4 · Q6
user_id	text K
channel_id	text C↑
joined_at	timestamp

misma membresía guardada desde los dos lados (ADR-06)

se escriben juntas con un BATCH logged: ambas terminan aplicándose, sin aislamiento

keyspace messaging
solo lo usa message-service

messages_by_channel	Q7 · Q8
channel_id	text K
day	date K
message_id	timeuuid C↓
author_id	text
author_name	text
content	text

partition key compuesta:
(channel_id, day) → un canal en un día (ADR-03)

clustering message_id DESC:
los más nuevos primero

author_name se copia en cada mensaje: no hay JOIN con users (desnormalización)

Cómo leer el modelo

- K** Partition key: decide en qué nodo vive la fila; toda consulta la debe dar completa.
- C↑ / C↓** Clustering key: orden de las filas dentro de la partición (ascendente / descendente).

- IDs naturales: user_id es el nombre de usuario y channel_id el nombre del canal. La unicidad se garantiza con INSERT ... IF NOT EXISTS (transacción ligera), que responde ALREADY_EXISTS si el nombre ya está tomado. Así no hacen falta tablas extra para buscar por nombre.
- Sin llaves foráneas: channel_id y author_id en messaging son referencias lógicas a community. La validez la garantiza message-service con IsMember (ADR-02).
- Q2 lee toda la tabla users: es aceptable porque la tabla es pequeña en la demo; con millones de usuarios haría falta paginar o buscar por prefijo.

Lecciones aprendidas

Partir por capacidad de negocio, no por tabla

En Cassandra la partición lo decide todo

Guardar mucho no es guardar íntegro

Base por servicio: la regla la garantiza el motor

Go: composición e interfaces, sin herencia

Mostrar dónde se rompe enseña más

Fuentes y referencias

Enlaces a las encuestas y reportes usados en la presentación

Lenguajes, bases de datos e IDEs

Stack Overflow Developer Survey 2025

Lenguajes, bases de datos e IDEs
survey.stackoverflow.co/2025/technology

JetBrains, Developer Ecosystem 2025

Go: lenguaje más buscado para adoptar
blog.jetbrains.com/research

JetBrains, Developer Ecosystem 2024

Tendencia de Go y Dart 2019–2024
jetbrains.com/lp/devecosystem-2024

GitHub Octoverse 2025

Go entre los 10 lenguajes más usados
github.blog/news-insights/octoverse

Uso de microservicios

CNCF y SlashData, Cloud Native 2025

46% de backend usa microservicios
cncf.io/announcements/2025/11/11

Gartner Peer Community

74% de organizaciones ya los usan (~2023)
gartner.com/peer-community

Ofertas laborales (fuentes débiles)

DEV Community, guía backend 2026

Palabras clave en 80%+ de las ofertas
dev.to/sarah_m

Talent500

~75% de roles backend piden cloud
talent500.com/blog

MoldStud

+70% de ofertas con microservicios
moldstud.com/articles

Glassdoor EE. UU.

2,337 vacantes (cifra cambia a diario)
glassdoor.com/Job

¿Preguntas?